

Модуль 9. Основи продуктивності та оптимізації

Урок 3. Швидкість API, HTTP-кешування та підготовка до highload

У **Уроці 9.1** ви навчилися мислити продуктивністю на рівні системи, знаходити вузькі місця та уникати передчасної оптимізації. У **Уроці 9.2** ви застосовували кешування та оптимізацію БД: eager loading, індекси, пагінацію, Redis, інвалідацію. Цей урок завершує **Модуль 9 і всю програму**: ми розглядаємо **швидкість API як продукт** - що впливає на час відповіді, як використовувати HTTP-кешування (Cache-Control, ETag), навіщо потрібні пагінація та rate limiting, як виносити важку логіку в jobs і як готувати застосунок до зростання навантаження (highload). Усі приклади - з **фінансової сфери**; код - **строго типізований**. Проєкти розгортаються через **docker-compose**. Після цього уроку ви матимете повний набір навичок: від архітектури та API через черги, тести, логування та моніторинг до свідомої оптимізації та production-ready API.

1. Мета уроку

Мета цього уроку - навчити оптимізувати **API як продукт**: швидкість відповіді, контроль об'єму даних, HTTP-кешування та захист від перевантаження.

Після уроку ви повинні:

- розуміти, з чого складається час відповіді API та де можлива оптимізація на кожному етапі;
- знати принципи HTTP-кешування (Cache-Control, ETag) та коли воно ефективне;
- вміти обмежувати об'єм даних (пагінація, DTO/Resources) та контролювати навантаження (rate limiting);
- усвідомлювати роль асинхронної обробки (jobs) для швидкої відповіді API;
- мислити підготовкою до highload: stateless API, кеш, передбачуване навантаження;
- бачити, як виглядає production-ready API в цілому.

2. API як вузьке місце системи

У сучасних застосунках від API залежать frontend, мобільний додаток, інтеграції та зовнішні сервіси. **Повільне API дорівнює повільній системі** незалежно від клієнта: якщо бекенд відповідає за 3 секунди, користувач чекає 3 секунди. Тому швидкість та стабільність API - пріоритет для всього продукту. У фінансовому застосунку це ще й питання довіри: постійні таймаути або «висипання» при піковому навантаженні підривають впевненість користувачів у системі. Модулі 7-8 дали вам контракт API, документацію, тести та моніторинг; Модуль 9 доповнює це **швидкістю, кешем та контролем навантаження**.

3. Що означає «швидке API»

Швидке API - це не лише «один запит виконався за 50 мс». Важливо: **стабільність** - час відповіді не «скаче» при зміні навантаження або об'єму даних; **передбачуваність** - p95 та p99 у межах прийнятних значень; **відсутність деградації** при зрості даних (пагінація, індекси, кеш); **коректна робота під навантаженням** - rate limiting та кеш захищають від перевантаження. Один «швидкий

запит» локально - не показник; орієнтуватися потрібно на метрики в продакшені (логи, APM, моніторинг з Уроку 8).

4. З чого складається час відповіді API

Типова структура обробки запиту: **(1)** прийом HTTP-запиту та мережева затримка; **(2)** middleware (аутентифікація, логи, correlation_id); **(3)** бізнес-логіка (контролер, сервіси); **(4)** робота з БД (запити, можливо кеш); **(5)** формування відповіді (серіалізація JSON, DTO/Resources); **(6)** відправка клієнту. Оптимізація можлива **на кожному етапі**: зменшити кількість запитів до БД та використати кеш (Урок 9.2); не повертати зайві поля та обмежити об'єм пагінацією; використовувати HTTP-кеш, щоб частина запитів взагалі не доходила до бізнес-логіки; виносити важку логіку в jobs. Найбільший виграш зазвичай дають БД та об'єм відповіді; HTTP-кешування зменшує навантаження на сервер при повторних запитах того самого ресурсу.

5. HTTP як рівень оптимізації

HTTP - це не лише транспорт для JSON. Він дає **механізми кешування** (Cache-Control, ETag, Last-Modified), **статус-коди** (200, 304 Not Modified), **заголовки** для контролю кешу та умовних запитів. Ігнорування цих можливостей означає втрату виграшу: клієнт або проксі можуть зберігати відповідь і не звертатися до сервера знову, якщо дані не змінилися. Для read-only публічних або рідко змінюваних ресурсів (тарифи, умови, довідники) HTTP-кешування значно знижує навантаження та прискорює відповіді для повторних запитів.

6. Що таке HTTP-кешування

HTTP-кешування - це кешування на рівні клієнта (браузер, мобільний додаток) або проміжного проксі (CDN, reverse proxy). Мета: **не виконувати запит повністю**, якщо відповідь не змінилася. Сервер повертає заголовки (Cache-Control, ETag, Last-Modified); клієнт при наступному запиті може надіслати умовний запит (If-None-Match, If-Modified-Since); якщо ресурс не змінився, сервер відповідає **304 Not Modified** без тіла - клієнт використовує збережену копію. Таким чином економляться час, трафік та навантаження на застосунок. HTTP-кеш ефективний для **GET**-запитів, **read-only** ресурсів, **публічних** або **рідко змінюваних** даних. Він **не підходить** для персоналізованих даних без спеціальної логіки (наприклад, «мої платежі» залежать від user_id і часто змінюються), для POST/PATCH/DELETE та для даних у реальному часі.

7. Cache-Control та ETag (оглядово)

Cache-Control - заголовок, який вказує, чи можна кешувати відповідь і як довго. Приклади: **Cache-Control: public, max-age=60** - кешувати публічно 60 секунд; **Cache-Control: private, max-age=300** - кешувати лише на клієнті 5 хвилин; **Cache-Control: no-store** - не кешувати. **ETag** - ідентифікатор версії відповіді (наприклад, хеш вмісту або версія сутності). При наступному запиті клієнт надсилає **If-None-Match: "etag-value"**; якщо ресурс не змінився, сервер повертає 304 без тіла. У Laravel можна встановлювати заголовки у Response; у Symfony - через Response або event listener. Для фінансового API типовий сценарій - короткий max-age для публічних довідників (валюти, тарифи); для персональних даних (платежі, баланс) - no-store або дуже короткий private max-age, щоб не показувати застарілі суми.

8. Контроль об'єму даних у відповіді

Навіть швидкий запит до БД стає повільним для користувача, якщо **відповідь дуже велика**: багато даних по мережі, довга серіалізація JSON, велике споживання пам'яті. Принцип: **повертати лише те, що потрібно**. DTO та API Resources (Модуль 7) контролюють структуру та набір полів - вони впливають не лише на архітектуру, а й на швидкість та розмір відповіді. Пагінація обмежує кількість записів у списку; відсутність пагінації при великих таблицях - типова причина повільних та нестабільних ендпоінтів. У фінансовому API: не повертати зайві поля платежу (наприклад, внутрішні технічні ідентифікатори для клієнта дашборду); для виписки завжди використовувати пагінацію з розумним лімітом на сторінку.

9. Пагінація як обов'язковий інструмент

API без пагінації для списків **не масштабується**: при зрості даних відповідь і час виконання зростають, з'являються таймаути та пікове навантаження на БД. Пагінація обмежує розмір відповіді (наприклад, 20–50 записів на сторінку), стабілізує API та робить навантаження передбачуваним. Типи: **page + per_page** (offset-based) - простий, зручний для UI; **cursor-based** - стабільніша при частих змінах даних (наприклад, курсор по **id** або **created_at**). Вибір залежить від типу даних та вимог. У Laravel - `paginate()`; у Symfony - Doctrine Paginator або курсор. Для production-ready API пагінація для будь-якого ендпоінта-списку - **must-have**.

10. Ліміти (rate limiting) як захист системи

Rate limiting обмежує кількість запитів від клієнта (за IP, за токеном, за ендпоінтом) за певний інтервал. Мета - **захистити систему** від зловживань, DDoS, некоректних клієнтів або витоків, що генерують тисячі запитів. Ліміти стабілизують навантаження та дають справедливий доступ усім клієнтам. Вони **не «шкода для клієнтів»**, а захист для всіх: без лімітів один клієнт може вивести API з ладу. Ліміти особливо потрібні для: публічних API; авторизаційних ендпоінтів (захист від підбору); важких операцій (експорт, звіти); зовнішніх інтеграцій з обмеженнями. У Laravel - throttle middleware; у Symfony - аналогічні механізми або бандли (наприклад, nelmio/api-doc). У відповіді повертають заголовки (X-RateLimit-Limit, X-RateLimit-Remaining) та статус 429 Too Many Requests при перевищенні.

11. Бізнес-логіка і швидкість API

Проблеми виникають, коли **складна логіка виконується синхронно в кожному запиті**: кожен клієнт чекає, поки порахується статистика або згенерується звіт. Рішення: **винесення в jobs** (Модуль 5) - API приймає запит, ставить завдання в чергу, відповідає «завдання прийнято» або повертає посилання на результат; **попередні обчислення** - нічні jobs рахують зведену статистику, API лише віддає готові дані; **кешування результатів** (Урок 9.2) - для частини запитів віддавати закешований результат. API тоді залишається швидким: час відповіді не залежить від важких обчислень.

12. Jobs як частина оптимізації API

Jobs дозволяють **розвантажити API**: прийняти запит, поставити завдання в чергу, **швидко відповісти** клієнту. Приклад у фінансовому контексті: експорт виписки за період. Синхронний варіант - генерація PDF або CSV у запиті, відповідь через 10–30 секунд, ризик таймаутів. Асинхронний варіант - API повертає 202 Accepted та **task_id**; клієнт періодично запитує статус або отримує посилання на файл, коли job завершиться. Таким чином API завжди відповідає швидко, а

важка робота виконується у фоні. У docker-compose воркер черги (Модуль 5) обробляє такі завдання окремо від веб-сервера.

13. Підготовка до highload: концептуально

Highload - це велика кількість користувачів, запитів та даних одночасно. Підготовка включає: **кешування** - зменшення навантаження на БД та повторних обчислень; **асинхронність** - важкі операції в чергах; **контроль об'єму** - пагінація, обмеження полів, rate limiting; **стабільні API-контракти** - передбачувана поведінка та розмір відповіді. Мислення не «прискорити один сервер», а **масштабувати горизонтально**: кілька екземплярів застосунку за балансувальником. Для цього API має бути **stateless** (не покладатися на локальний стан сесії на конкретному сервері), передбачуваним та добре кешованим. Логи та метрики (Модуль 8) допомагають виявляти вузькі місця при зростанні навантаження.

14. Горизонтальне масштабування

При highload зазвичай **додають інстанси** застосунку (горизонтальне масштабування), а не лише «підсилюють» один сервер. Щоб це працювало: застосунок має бути **stateless** - аутентифікація по токєну, дані сесії в Redis або БД, а не в пам'яті одного процесу; **кеш та черги** - зовні (Redis), щоб усі інстанси працювали з одним кешем і однією чергою; **пагінація та ліміти** - щоб один клієнт не створював пікового навантаження. Тоді балансувальник розподіляє запити між інстансами, а логи та error tracking показують стан усієї системи. Деталі DevOps та інфраструктури виходять за рамки курсу; тут важливо **архітектурне мислення**: API готовий до масштабування, якщо він stateless, кешований і з контролем об'єму.

15. Типові помилки при «підготовці до highload»

Уникайте: **оптимізації без вимірювань** - складні рішення до виявлення реальних вузьких місць; **складних рішень без потреби** - мікросервіси та розподілений кеш «на майбутнє» замість простого eager loading та Redis; **ігнорування HTTP-кешу** - не використовувати Cache-Control для публічних ресурсів; **відсутності лімітів** - API без rate limiting вразливий до зловживань; **повернення «всього і одразу»** - списки без пагінації, великі JSON. Production-ready підхід: вимірювання, прості та перевірені інструменти (кеш, пагінація, jobs, throttle), потім поступове ускладнення за потреби.

16. Як виглядає production-ready API

Production-ready API в контексті програми: **швидке** - оптимізація БД, кеш, обмеження об'єму відповіді; **стабільне** - пагінація, індекси, передбачуваний час відповіді; **кешоване** - серверний кеш (Redis) та HTTP-кеш де доречно; **з пагінацією** для всіх списків; **з лімітами** - rate limiting для захисту та справедливого доступу; **з асинхронною логікою** - важкі операції в jobs; **з прогнозованим навантаженням** - моніторинг та логи для виявлення проблем. Разом з контрактом API (Модуль 7), тестами, логуванням та error tracking (Модуль 8) це дає повний цикл розробки та експлуатації бекенду.

17. Приклад коду: HTTP-заголовки кешу та пагінація

Laravel - контролер з Cache-Control та пагінацією:

```

<?php

declare(strict_types=1);

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use App\Repositories\PaymentRepository;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

final class AccountPaymentsController extends Controller
{
    public function __construct(
        private PaymentRepository $payments,
    ) {}

    /**
     * Список платежів рахунку. Пагінація + короткий private cache для
     * зменшення повторних запитів.
     */
    public function index(Request $request, int $accountId): JsonResponse
    {
        $perPage = min((int) $request->get('per_page', 20), 50);
        $paginator = $this->payments->getByAccountIdPaginated($accountId,
            $perPage);

        $response = response()->json([
            'data' => $paginator->items(),
            'meta' => [
                'current_page' => $paginator->currentPage(),
                'per_page' => $paginator->perPage(),
                'total' => $paginator->total(),
            ],
        ]);

        // Короткий кеш лише на клієнті (private), щоб не показувати
        // застарілі дані довго.
        $response->header('Cache-Control', 'private, max-age=15');

        return $response;
    }
}

```

Symfony - аналог з Cache-Control:

У Symfony те саме: формувати Response з JSON, встановити заголовок **Cache-Control**: `$response->headers->set('Cache-Control', 'private, max-age=15');`. Для публічного довідника (наприклад, валют) можна використати **public, max-age=3600**.

18. Приклад коду: rate limiting (Laravel)

У Laravel throttle налаштовується в **RouteServiceProvider** або в middleware для маршрутів:

```
// routes/api.php
Route::middleware(['auth:sanctum', 'throttle:60,1'])->group(function () {
    Route::get('accounts/{account}/payments',
        [AccountPaymentsController::class, 'index']);
    Route::post('payments', [PaymentsController::class, 'store']);
});
```

throttle:60,1 - 60 запитів на хвилину на користувача. При перевищенні Laravel повертає 429; клієнт отримує заголовки X-RateLimit-*. Для ендпоінтів експорту або важких звітів можна задати окремий, більш жорсткий ліміт (наприклад, 5 запитів на хвилину).

19. Завершення програми: що ви маєте в арсеналі

Після Модуля 9 ви пройшли повний цикл: **Модуль 5** - черги та асинхронність (jobs, workers, docker-compose); **Модуль 6** - структура коду (Controller, Service, Repository), тестування; **Модуль 7** - API (REST, DTO, валідація, документація, Feature-тести); **Модуль 8** - логування, моніторинг, error tracking (Sentry), лог-агрегація, інциденти; **Модуль 9** - продуктивність: профайлінг, кешування, оптимізація БД (eager loading, індекси), швидкість API, HTTP-кеш, пагінація, rate limiting, підготовка до highload. Разом це дає основи для розробки та підтримки **production-ready** бекенду у фінансовій та інших доменах.

20. Практична частина уроку

Усі завдання - у **фінансовому** контексті.

Завдання на уроці

1. HTTP-кешування та пагінація

Виберіть 2–3 API-ендпоінти у проєкті (наприклад, список платежів, довідник валют, умови тарифів). Для кожного визначте: чи потрібне HTTP-кешування (GET, read-only, рідко змінювані?); якщо так - який Cache-Control доречний (public/private, max-age); чи є пагінація для списків і чи достатня вона.

2. Ліміти

Для тих самих або інших ендпоінтів визначте: чи потрібні rate limits; для яких ендпоінтів обмеження мають бути жорстнішими (наприклад, експорт виписки, створення платежу). Опишіть або налаштуйте throttle (Laravel) / аналог (Symfony) з розумними значеннями (наприклад, 60 req/min для звичайних, 5 req/min для експорту).

3. Об'єм відповіді та зайві поля

Проаналізуйте один ендпоінт, що повертає список: який об'єм даних у відповіді (кількість полів, розмір JSON); чи є зайві поля для клієнта. Запропонуйте зміни (DTO/Resource з обмеженням полів, пагінація), щоб зменшити розмір та стабілізувати час відповіді.

4. Винесення логіки в jobs

Виберіть один сценарій важкої операції (експорт виписки, генерація звіту по рахунках, масове

оновлення). Опишіть, як перевести його на асинхронну модель: API приймає запит, ставить job, повертає 202 та task_id; клієнт перевіряє статус або отримує результат по посиланню. Оцініть очікуваний ефект: зменшення часу відповіді API, зникнення таймаутів для цієї операції.

5. Підсумок production-ready API

Сформулюйте для свого проекту (або уявного фінансового API): які з перелічених елементів уже є (пагінація, кеш, ліміти, jobs); що варто додати в першу чергу; як це пов'язано з моніторингом та логами (Модуль 8) - які метрики допоможуть перевірити, що API дійсно швидке та стабільне під навантаженням.